



Universidad
Tecnológica
del Perú

PRIMERA UNIDAD DE APRENDIZAJE 2: Patrones creacionales

CURSO: DISEÑO DE PATRONES

SESIÓN N° 07:

**Patrones Creacionales:
Patrón Factory. Patrón Abstract Factory. Patrón
Abstract Builder.**

DOCENTES DE UTP
SISTEMAS



Lo que debemos tener en cuenta



5 minutos



¿QUÉ SON PATRONES Abstract Factory y Factory?

https://youtu.be/izFGKo7Z1lg?si=Mb80kFXQg_Zk9cca



¿Qué vimos en la sesión anterior?



El patrón Singleton asegura que una clase tenga una única instancia y proporciona un punto de acceso _____ a esa instancia. En el patrón Prototype, los objetos son creados mediante la clonación de un _____ existente. El patrón Singleton es útil en situaciones donde se necesita controlar el acceso a algún _____ compartido. Una desventaja del patrón Singleton es que puede ser difícil de _____ en sistemas multihilo. El patrón Prototype permite la creación de nuevos objetos a partir de un _____ sin necesidad de conocer su clase concreta.

☐ Show answers

probar

global

recurso

modelo

prototype



Unidad de aprendizaje 2:
Patrones creacionales.

Semana 5,6 y 7

Logro específico de aprendizaje:

Al finalizar la unidad el participante aplica los patrones creacionales para la solución de problemas.

Temario:

- Consolidación de temas (Repaso).
- Patrones Creacionales: Introducción a patrones creacionales. Patrón Singleton. Patrón Prototype.
- Patrones Creacionales: Patrón Factory. Patrón Abstract Factory. Patrón Abstract Builder.

Al final de la sesión el estudiante aplica patrones creacionales:

Patrón Factory. Patrón Abstract Factory. Patrón Abstract Builder en un contexto definido



Un patrón de diseño de fábrica es una forma de patrón de diseño de creación que nos permite crear un objeto para instanciarlo mediante una interfaz o una clase. La fábrica es el método más eficiente para crear un objeto. Los objetos se crean mediante este método sin proporcionarle el razonamiento al cliente. El cliente utiliza la misma interfaz estándar para crear un nuevo tipo de objeto.



¡Que no se te escape nada de tu clase!:

SESIÓN N° 07:

**Patrones Creacionales:
Patrón Factory. Patrón Abstract Factory. Patrón
Abstract Builder.**

El patrón Factory es un patrón de diseño de creación, lo que significa que se ocupa de la creación de objetos.

En el diseño Factory, generamos objetos sin revelar el mecanismo de creación al cliente, y el cliente crea nuevos tipos de objetos utilizando la misma interfaz estándar.

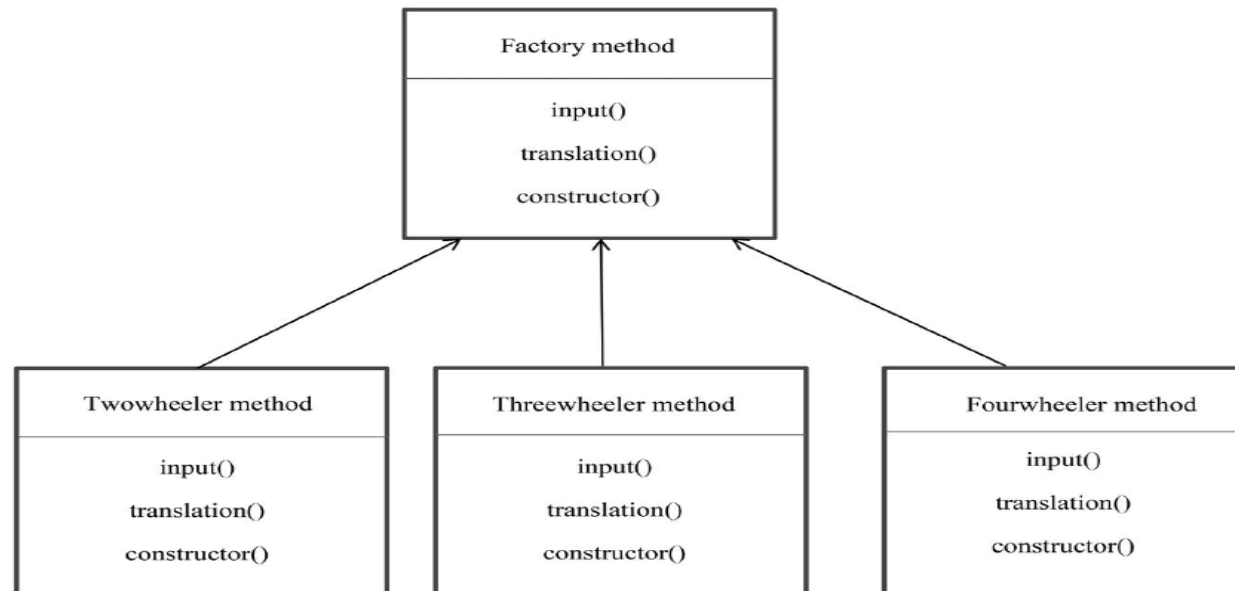
El objetivo es emplear una función miembro estática (patrón Factory estático) que crea y devuelve instancias mientras mantiene oculta la información del módulo de clase para el usuario.



Un patrón de fábrica es una de las ideas de diseño clave para crear un objeto, que permite a los clientes generar objetos de biblioteca de una manera que no esté estrechamente vinculada con la jerarquía de clases de la biblioteca.

¿A qué nos referimos exactamente cuando hablamos de "biblioteca" y "clientes"?

Una biblioteca es proporcionada por un tercero y expone algunas API públicas a las que los clientes realizan llamadas para realizar sus trabajos. Los diferentes tipos de vistas que ofrece el sistema operativo Android son un ejemplo sencillo.



Su principio es similar al polimorfismo en el sentido de que no se requieren modificaciones en el código del cliente. Supongamos que deseamos dibujar varias formas, como rectángulos, cuadrados, círculos, etc. El método Factory Pattern puede usarse para crear la instancia en función de la entrada del usuario. Podemos reservar un vehículo de una rueda, de dos ruedas, de tres ruedas y de cuatro ruedas en una aplicación de taxi. El cliente puede reservar cualquiera de los viajes que desee desde esta página. Podemos construir una clase llamada Booking con la ayuda de la función Factory, que nos permitirá generar una instancia que acepte la entrada del usuario. Como resultado, el desarrollador no necesita alterar el código completo para implementar la nueva función. El método Factory elimina el código lógico complejo que es difícil de conservar. También nos impide realizar cambios en la base de código, ya que alterar el código existente puede producir fallas sutiles y hacer que cambie el comportamiento.

Supongamos que queremos crear un sitio web que venda libros en varias partes del país.

La primera edición del sitio web solo acepta pedidos de libros, pero a medida que pase el tiempo y nuestro sitio web crezca en popularidad, no agregaremos cosas adicionales a las ventas, como ropa y calzado.

Es un concepto excelente, pero ¿qué pasa con los desarrolladores de software? Ahora deben actualizar todo el código base porque la mayor parte del código está relacionado con la clase del libro y deben alterar todo el código base. Puede resultar en un código desordenado.

En lugar de utilizar la construcción de objetos sencilla, utilizamos el patrón Factory específico en la solución para invocar el objeto de construcción.

Ambos métodos de producción de objetos son bastante similares, pero se hace referencia a ellos dentro de la función Factory.

Por ejemplo, nuestros productos de venta, como libros, teléfonos móviles, ropa y accesorios, deberían tener una interfaz de compra que declare un método de compra. Estos métodos se implementarán de forma diferente en cada clase.



Patrones creacionales – Factory - ejemplo

Paso 1: Definir la interfaz del producto

// Interfaz de Notificación

```
public interface Notificacion {  
    void enviarMensaje(String mensaje);  
}
```

Paso 2: Crear implementaciones concretas del producto

// Implementación concreta de Notificación por Correo Electrónico

```
public class NotificacionCorreo implements Notificacion {  
    @Override  
    public void enviarMensaje(String mensaje) {  
        System.out.println("Enviando correo electrónico: " + mensaje);  
    }  
}
```

// Implementación concreta de Notificación por SMS

```
public class NotificacionSMS implements Notificacion {  
    @Override  
    public void enviarMensaje(String mensaje) {  
        System.out.println("Enviando SMS: " + mensaje);  
    }  
}
```

Paso 3: Crear la Fábrica

// Fábrica de Notificaciones

```
public class NotificacionFactory {  
    public static Notificacion  
    crearNotificacion(String tipo) {  
        if (tipo.equalsIgnoreCase("correo")) {  
            return new NotificacionCorreo();  
        } else if (tipo.equalsIgnoreCase("sms")) {  
            return new NotificacionSMS();  
        } else {  
            throw new  
            IllegalArgumentException("Tipo de  
            notificación desconocido");  
        }  
    }  
}
```


Patrones creacionales – Factory - ejemplo

Paso 4: Usar la Fábrica en el Cliente

```
public class Cliente {  
    public static void main(String[] args) {  
        // Crear notificación por correo  
        Notificacion notificacionCorreo = NotificacionFactory.crearNotificacion("correo");  
        notificacionCorreo.enviarMensaje("Hola por correo!");  
  
        // Crear notificación por SMS  
        Notificacion notificacionSMS = NotificacionFactory.crearNotificacion("sms");  
        notificacionSMS.enviarMensaje("Hola por SMS!");  
    }  
}
```

Salida esperada

Enviando correo electrónico: Hola por correo!

Enviando SMS: Hola por SMS!

El patrón Abstract Factory es un patrón de diseño creacional que proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas. Este patrón es útil cuando el sistema debe ser independiente de cómo sus productos son creados, compuestos y representados.

Estructura

- ****Abstract Factory****: Declara una interfaz para la creación de productos abstractos.
 - ****Concrete Factory****: Implementa la interfaz de la fábrica abstracta para crear instancias de productos concretos.
 - ****Abstract Product****: Declara una interfaz para un tipo de producto.
 - ****Concrete Product****: Implementa la interfaz del producto abstracto.
 - ****Client****: Usa solo las interfaces declaradas por las fábricas abstractas y los productos abstractos.
-
-

Las clases abstractas de Factory suelen crearse con Factory Methods, aunque también pueden realizarse mediante Prototype.

Los Factory Methods suelen invocarse desde dentro de Template Methods.

La herencia se utiliza para crear objetos en el Factory Method. La delegación se utiliza para crear prototipos.

Normalmente, los diseños comienzan con el Factory Method (menos difícil, más adaptable, proliferan las subclases) y progresan hacia el Abstract Factory, Prototype o Builder (más flexible, más complejo) cuando el diseñador se da cuenta de que se requiere más flexibilidad.

Aunque la creación de prototipos no necesita subclasificación, sí requiere una acción de inicialización. El Factory Method necesita subclasificación pero no necesita inicializar.

Un Factory Method ofrece la ventaja de devolver repetidamente la misma instancia o de devolver una subclase en lugar de un objeto del mismo tipo.

Patrones creacionales – Abstract Factory - ejemplo

Paso 1: Definir interfaces de los productos

```
// Interfaz para Silla
public interface Silla {
    void sentarse();
}
```

```
// Interfaz para Sofá
public interface Sofa {
    void acostarse();
}
```

Paso 2: Crear implementaciones concretas de los productos

// Implementación concreta de Silla Moderna

```
public class SillaModerna implements Silla {
    @Override
    public void sentarse() {
        System.out.println("Sentado en una silla moderna");
    }
}
```

// Implementación concreta de Sofá Moderno

```
public class SofaModerno implements Sofa {
    @Override
    public void acostarse() {
        System.out.println("Acostado en un sofá moderno");
    }
}
```

// Implementación concreta de Silla Victoriana

```
public class SillaVictoriana implements Silla {
    @Override
    public void sentarse() {
        System.out.println("Sentado en una silla victoriana");
    }
}
```

// Implementación concreta de Sofá Victoriano

```
public class SofaVictoriano implements Sofa {
    @Override
    public void acostarse() {
        System.out.println("Acostado en un sofá victoriano");
    }
}
```

Patrones creacionales – Abstract Factory - ejemplo

Paso 3: Definir la interfaz de la fábrica abstracta

// Interfaz para la fábrica de muebles

```
public interface MueblesFactory {  
    Silla crearSilla();  
    Sofa crearSofa();  
}
```

Paso 4: Crear las implementaciones concretas de las fábricas

// Fábrica de Muebles Modernos

```
public class MueblesModernosFactory implements MueblesFactory {  
    @Override  
    public Silla crearSilla() {  
        return new SillaModerna();  
    }  
    @Override  
    public Sofa crearSofa() {  
        return new SofaModerno();  
    }  
}
```

// Fábrica de Muebles Victorianos

```
public class MueblesVictorianosFactory implements  
MueblesFactory {  
    @Override  
    public Silla crearSilla() {  
        return new SillaVictoriana();  
    }  
    @Override  
    public Sofa crearSofa() {  
        return new SofaVictoriano();  
    }  
}
```

Patrones creacionales – Abstract Factory - ejemplo

Paso 5: Usar la fábrica abstracta en el cliente

```
public class Cliente {  
    private Silla silla;  
    private Sofa sofa;  
    public Cliente(MueblesFactory factory) {  
        silla = factory.crearSilla();  
        sofa = factory.crearSofa();  
    }  
    public void usarMuebles() {  
        silla.sentarse();  
        sofa.acostarse();  
    }  
    public static void main(String[] args) {  
        MueblesFactory factoryModernos = new MueblesModernosFactory();  
        Cliente clienteModernos = new Cliente(factoryModernos);  
        clienteModernos.usarMuebles();  
        MueblesFactory factoryVictorianos = new MueblesVictorianosFactory();  
        Cliente clienteVictorianos = new Cliente(factoryVictorianos);  
        clienteVictorianos.usarMuebles();  
    }  
}
```

Salida esperada

...

Sentado en una silla moderna
Acostado en un sofá moderno
Sentado en una silla victoriana
Acostado en un sofá victoriano
...

Patrones creacionales – Builder

Paso 1: Crear la clase `Coche` con su `Builder`

```
public class Coche {  
    // Atributos del coche  
    private String marca;  
    private String modelo;  
    private int año;  
    private String color;  
    private boolean tieneAireAcondicionado;  
    private boolean tieneGPS;  
    // Constructor privado  
    private Coche(CocheBuilder builder) {  
        this.marca = builder.marca;  
        this.modelo = builder.modelo;  
        this.año = builder.año;  
        this.color = builder.color;  
        this.tieneAireAcondicionado = builder.tieneAireAcondicionado;  
        this.tieneGPS = builder.tieneGPS;  
    }  
}
```

Patrón es útil cuando quieres construir un objeto complejo paso a paso y quieres evitar constructores telescópicos (constructores con muchos parámetros). Supongamos que queremos construir un objeto `Coche` que tiene varios atributos como `marca`, `modelo`, `año`, `color`, etc.

Patrones creacionales – Builder

```
// Clase estática Builder
public static class CocheBuilder {
    private String marca;
    private String modelo;
    private int año;
    private String color;
    private boolean tieneAireAcondicionado;
    private boolean tieneGPS;

    // Métodos setter que retornan el propio Builder
    public CocheBuilder setMarca(String marca) {
        this.marca = marca;
        return this;
    }
    public CocheBuilder setModelo(String modelo) {
        this.modelo = modelo;
        return this;
    }
}
```

```
public CocheBuilder setAño(int año) {
    this.año = año;
    return this;
}

public CocheBuilder setColor(String color) {
    this.color = color;
    return this;
}

public CocheBuilder setTieneAireAcondicionado(boolean
tieneAireAcondicionado) {
    this.tieneAireAcondicionado = tieneAireAcondicionado;
    return this;
}

public CocheBuilder setTieneGPS(boolean tieneGPS) {
    this.tieneGPS = tieneGPS;
    return this;
}
```


Patrones creacionales – Builder

```
// Método build que retorna el objeto final Coche
```

```
public Coche build() {  
    return new Coche(this);  
}  
}
```

```
@Override
```

```
public String toString() {  
    return "Coche [marca=" + marca + ", modelo=" + modelo + ", año=" + año + ", color=" + color  
        + ", tieneAireAcondicionado=" + tieneAireAcondicionado + ", tieneGPS=" + tieneGPS + "];"  
}  
}
```

Patrones creacionales – Builder

Paso 2: Usar el Builder para crear un objeto `Coche`

```
```java
public class Cliente {
 public static void main(String[] args) {
 Coche coche = new Coche.CocheBuilder()
 .setMarca("Toyota")
 .setModelo("Corolla")
 .setAño(2021)
 .setColor("Rojo")
 .setTieneAireAcondicionado(true)
 .setTieneGPS(true)
 .build();

 System.out.println(coche);
 }
}
```

### Salida esperada

```
```
Coche [marca=Toyota, modelo=Corolla, año=2021,
color=Rojo, tieneAireAcondicionado=true, tieneGPS=true]
```
```

Este ejemplo muestra cómo el patrón Builder se utiliza para construir un objeto `Coche` de manera clara y concisa, evitando constructores con muchos parámetros y mejorando la legibilidad del código.

# ¿Cuál es la diferencia entre Factory y Builder?



# Caso práctico



Realizar ejercicio práctico es una excelente manera de aprender y entender cómo elaborar un patrón de diseño creacional. Aquí tienes un ejercicio práctico detallado que puedes seguir:

### Ejercicio Práctico:

#### Caso de Uso: Plataforma de Pago en Línea

**Una plataforma de pago en línea necesita procesar diferentes métodos de pago, como tarjetas de crédito, PayPal y criptomonedas. Cada método de pago requiere una lógica específica para su procesamiento.** Se utiliza el patrón Factory para encapsular la creación de diferentes tipos de procesadores de pago. La plataforma decide cuál procesador de pago utilizar en función del método de pago seleccionado por el usuario

#### Objetivos:

1. Elaborar el código java utilizando el patrón seleccionado.



¿Quién quisiera participar? (2 voluntarios)







¿Qué se les hizo más fácil?  
¿Qué se les hizo más retador?





**¿Qué hemos aprendido el día de hoy?**



Tomar apuntes de manera eficaz ayuda a consolidar el aprendizaje y prepararse para los exámenes.



## 1. Propósito y Uso

- **\*\*Factory\*\***: El patrón Factory Method se utiliza principalmente para crear objetos de una jerarquía de clases relacionados mediante una interfaz común, ocultando el tipo específico de objeto que se crea. Es ideal cuando el proceso de creación de objetos puede variar y se desea encapsular esta lógica en una clase separada.
  - **\*\*Ejemplo\*\***: Crear diferentes tipos de notificaciones (correo electrónico, SMS) en función de una entrada del usuario.
- **\*\*Builder\*\***: El patrón Builder se utiliza para construir objetos complejos paso a paso. Es útil cuando un objeto tiene muchas opciones de configuración y es necesario un enfoque más flexible y controlado para la creación de instancias.
  - **\*\*Ejemplo\*\***: Construir un objeto `Coche` con múltiples atributos opcionales como `marca`, `modelo`, `año`, `color`, etc.



## 2. Flexibilidad en la Creación

- **\*\*Factory\*\***: La flexibilidad se centra en qué tipo de objeto se crea, permitiendo la creación de una familia de objetos relacionados sin especificar la clase concreta. La lógica de creación de objetos puede estar centralizada y modificarse fácilmente.
- **\*\*Ventaja\*\***: Facilita la creación de nuevos tipos de objetos sin modificar el código cliente.
- **\*\*Builder\*\***: La flexibilidad se centra en cómo se crea el objeto, permitiendo una construcción detallada y paso a paso. Se puede personalizar cada paso del proceso de construcción sin necesidad de múltiples constructores sobrecargados.
- **\*\*Ventaja\*\***: Facilita la creación de objetos complejos con múltiples configuraciones opcionales, mejorando la legibilidad y mantenibilidad del código.



## 3. Complejidad y Legibilidad del Código

- **\*\*Factory\*\***: Simplifica la creación de objetos al encapsular la lógica de creación en una fábrica, pero puede introducir complejidad si hay demasiados tipos de productos o si la lógica de creación es muy variada.
  - **\*\*Desventaja\*\***: Puede resultar en una jerarquía de clases más compleja con múltiples fábricas si no se gestiona adecuadamente.
- **\*\*Builder\*\***: Simplifica la construcción de objetos complejos proporcionando una interfaz clara y fluida para configurar las opciones del objeto. Esto mejora la legibilidad y la mantenibilidad del código, especialmente cuando el objeto tiene muchos atributos opcionales.
  - **\*\*Desventaja\*\***: Puede ser excesivo para objetos simples donde un patrón Factory o incluso un simple constructor serían suficientes.



- Elabora la actividad práctica de acuerdo a la guía de laboratorio de sesión
- Sube la actividad práctica en la plataforma virtual de aprendizaje
- Guarda la actividad con la siguiente etiqueta:  
***DPA\_Actividad07\_NombreApellido***

*Nota: No olvides también revisar tu plataforma UTP+Class*





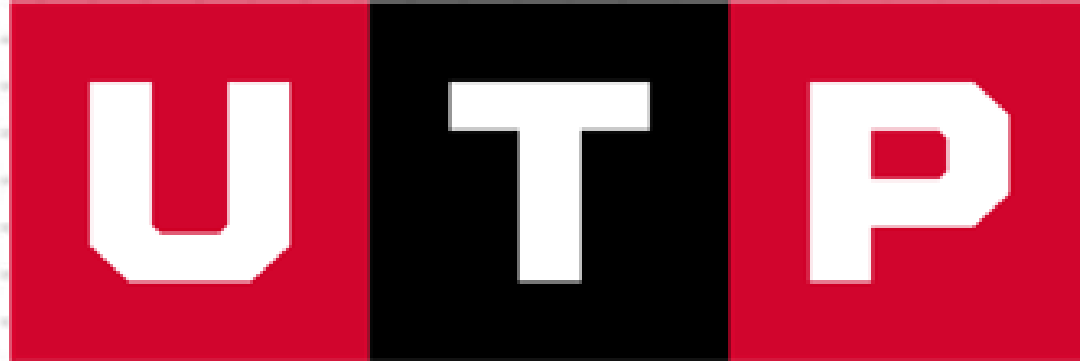
- Abstract Factory VS Factory / Concepto básico (2023). Coding Together ES.<https://youtu.be/izFGKo7Z1lg?si=bWORSurulo27k2MR>
- bin Uzayr, S. (2023). Software Design Patterns: The Ultimate Guide. CRC Press.

*Nota: No olvides también revisar tu plataforma UTP+Class*



**MUCHAS GRACIAS QUE  
DIOS LOS BENDIGA!!!**





**Universidad  
Tecnológica  
del Perú**

